Stage 3.3 — Environment-Specific YAML Configuration

Let's implement Stage 3.3 — Environment-Specific YAML Configuration directly in the project you've already built. The goal is to make your configuration behave differently for development, testing, and production without changing your Python application code.

1. What are we trying to achieve?

Currently you have one set of configuration values:

```
config/
├── app.yaml
├── llm.yaml
├── embedding.yaml
├── vector_database.yaml
├── retrieval.yaml
├── chunking.yaml
└── logging.yaml
```

Suppose your development environment uses:

```yaml
llm:
  model: llama-3.1-8b-instant

retrieval:
  top_k: 5
```

But production needs:

```yaml
llm:
  model: production-model

retrieval:
  top_k: 10
```

We don't want to modify Python code when moving to production.
Instead:

2. Our new project structure

We'll modify your existing project to:

```
YAML-Stage3.2-Pydantic-Settings/
│
├── config/
│   │
│   ├── base.yaml
│   ├── development.yaml
│   ├── testing.yaml
│   └── production.yaml
│
├── src/
│   ├── __init__.py
│   ├── yaml_loader.py
│   ├── config_loader.py
│   └── settings.py
│
└── main.py
```

Important change

Previously:

```
config/
├── app.yaml
├── llm.yaml
├── embedding.yaml
...
```

Now we're grouping configuration by environment.

3. Create base.yaml

File: config/base.yaml

Put the common configuration here.

```
application:
  name: Enterprise RAG application
  version: "1.0"

llm:
  provider: Groq
  temperature: 0

embedding:
  provider: sentence-transformers
  model: all-MiniLM-L6-v2

vector_database:
  provider: chroma
  directory: ./chroma_db
  collection_name: enterprise_knowledge_base

retrieval:
  reranking: false

chunking:
  strategy: recursive
  chunk_size: 1000
  chunk_overlap: 200

logging:
  level: INFO
  file: logs/application.log
```

Notice that we haven't specified:

```
application:
  environment:
```

or:

```
llm:
  model:
```

Those can be environment-specific.

4. Create development.yaml

File: config/development.yaml

```yaml
application:
  environment: development

llm:
  model: llama-3.1-8b-instant

retrieval:
  top_k: 5
```

So development inherits the common configuration from base.yaml and adds/overrides these values.

5. Create testing.yaml

File: config/testing.yaml

```yaml
application:
  environment: testing

llm:
  model: llama-3.1-8b-instant

retrieval:
  top_k: 3
```

Testing can use a smaller retrieval value.

6. Create production.yaml

File: config/production.yaml

```yaml
application:
  environment: production

llm:
  model: llama-3.1-8b-instant

retrieval:
  top_k: 10

logging:
  level: WARNING
```

Notice something important.
Production overrides:

```yaml
logging:
  level: WARNING
```

while the base configuration says:

```yaml
logging:
  level: INFO
```

Therefore production should ultimately get:

```
WARNING
```

This is our first example of a configuration override.

7. Remove the old YAML files

You can now remove the old:

```
app.yaml
llm.yaml
embedding.yaml
vector_database.yaml
retrieval.yaml
chunking.yaml
logging.yaml
```

because their configuration has been reorganized into:

```
base.yaml
development.yaml
testing.yaml
production.yaml
```

Your directory should now look like:

```
config/
├── base.yaml
├── development.yaml
├── testing.yaml
└── production.yaml
```

8. The important part — modify config_loader.py

Now we need to teach our loader about environments.

File: src/config_loader.py

Replace the current implementation with:

```
from pathlib import Path
from typing import Any

import yaml


def load_yaml(file_path: Path) -> dict[str, Any]:
    """
    Load one YAML file and return its contents
    as a Python dictionary.
    """

    with open(file_path, "r", encoding="utf-8") as file:

        # Convert YAML into a Python dictionary.
        config = yaml.safe_load(file)

    # Return an empty dictionary if the YAML file is empty.
    return config or {}


def merge_configs(
    base_config: dict[str, Any],
    override_config: dict[str, Any],
) -> dict[str, Any]:
    """
    Merge override_config into base_config.

    Nested dictionaries are merged recursively.
    """

    # Loop through every key in the override configuration.
    for key, value in override_config.items():

        # If both values are dictionaries,
        # merge them recursively.
        if (
            key in base_config
            and isinstance(base_config[key], dict)
            and isinstance(value, dict)
        ):
            merge_configs(base_config[key], value)

        else:
            # Otherwise the environment-specific value
            # replaces the base value.
            base_config[key] = value

    return base_config


def load_environment_config(
    config_dir: Path,
    environment: str,
) -> dict[str, Any]:
    """
    Load base.yaml and merge the selected
    environment configuration.
    """

    # Locate base configuration.
```

```
    base_file = config_dir / "base.yaml"

    # Locate environment-specific configuration.
    environment_file = config_dir / f"{environment}.yaml"

    # Make sure base.yaml exists.
    if not base_file.exists():
        raise FileNotFoundError(
            f"Base configuration not found: {base_file}"
        )

    # Make sure the requested environment exists.
    if not environment_file.exists():
        raise FileNotFoundError(
            f"Environment configuration not found: "
            f"{environment_file}"
        )

    # Load base configuration.
    base_config = load_yaml(base_file)

    # Load environment-specific configuration.
    environment_config = load_yaml(environment_file)

    # Merge environment configuration over base configuration.
    return merge_configs(
        base_config,
        environment_config,
    )
```

9. Understand the new merge_configs() function

This is one of the most important concepts in today's lesson.

Suppose base.yaml contains:

```
retrieval:
  top_k: 5
  reranking: false
```

And production contains:

```
retrieval:
  top_k: 10
```

Our merge operation produces:

```
retrieval:
  top_k: 10
  reranking: false
```

Why?

Because:

```
base
 │
 ├── top_k: 5
 └── reranking: false

production
 │
 └── top_k: 10
```

becomes:

```
final
 │
 ├── top_k: 10       ← overridden
 └── reranking: false ← inherited
```

This is exactly what we want.

10. Modify settings.py

Now we need to tell our Settings class which environment to load.

File: src/settings.py

The Pydantic models can remain almost exactly as they are.

The important changes are at the bottom.

Use:

```
from pathlib import Path

from pydantic import BaseModel

from src.config_loader import load_environment_config


# ============================================================
# Application Settings
# ============================================================

class ApplicationSettings(BaseModel):

    name: str
    version: str
    environment: str


# ============================================================
# LLM Settings
# ============================================================

class LLMSettings(BaseModel):

    provider: str
    model: str
    temperature: float


# ============================================================
# Embedding Settings
# ============================================================

class EmbeddingSettings(BaseModel):

    provider: str
    model: str


# ============================================================
# Vector Database Settings
# ============================================================

class VectorDatabaseSettings(BaseModel):

    provider: str
    directory: str
    collection_name: str


# ============================================================
# Retrieval Settings
# ============================================================

class RetrievalSettings(BaseModel):

    top_k: int
    reranking: bool


# ============================================================
```

```
# Chunking Settings
# ============================================================

class ChunkingSettings(BaseModel):

    strategy: str
    chunk_size: int
    chunk_overlap: int


# ============================================================
# Logging Settings
# ============================================================

class LoggingSettings(BaseModel):

    level: str
    file: str


# ============================================================
# Main Settings
# ============================================================

class Settings(BaseModel):

    application: ApplicationSettings
    llm: LLMSettings
    embedding: EmbeddingSettings
    vector_database: VectorDatabaseSettings
    retrieval: RetrievalSettings
    chunking: ChunkingSettings
    logging: LoggingSettings


# ============================================================
# Project directories
# ============================================================

BASE_DIR = Path(__file__).resolve().parent.parent

CONFIG_DIR = BASE_DIR / "config"


# ============================================================
# Select environment
# ============================================================

# Change this value to test different environments.
ENVIRONMENT = "development"


# ============================================================
# Load environment-specific configuration
# ============================================================

_CONFIG = load_environment_config(
    CONFIG_DIR,
    ENVIRONMENT,
)
```

```
# ============================================================
# Create validated Settings object
# ============================================================

settings = Settings(**_CONFIG)
```

11. Test development

Your ENVIRONMENT currently says:

```
ENVIRONMENT = "development"
```

Run:

```
python main.py
```

Your output should contain:

```
Environment: development
LLM Model: llama-3.1-8b-instant
Top K: 5
```

12. Test production

Now change:

```
ENVIRONMENT = "development"
```

to:

```
ENVIRONMENT = "production"
```

Run:

```
python main.py
```

You should get:

```
Environment: production
LLM Model: llama-3.1-8b-instant
Top K: 10
Log Level: WARNING
```

Notice what happened.
We didn't change:

```
main.py
```

or:

```
LLM code
```

or:

```
retrieval code
```

We only changed the environment.

13. But there is an even better approach

Right now we have:

```
ENVIRONMENT = "development"
```

inside settings.py.
That's not ideal.
Why?
Because to deploy production, we'd have to modify the Python file.
Instead, eventually we want:

```
python main.py development
```

or:

```
python main.py production
```

Then Python determines the environment dynamically.
We'll implement that as the next refinement of Stage 3.3.

14. The architecture we've achieved

You've now evolved your configuration system significantly:

Earlier

```
settings.py
   ↓
hardcoded values
```

Then

```
YAML
   ↓
dictionary
   ↓
settings.py
```

Then

```
YAML
  ↓
dictionary
  ↓
Pydantic
  ↓
Settings object
```

Now

This is starting to look like a real configuration layer rather than simply a YAML exercise.

15. Your practical exercise

Don't just copy the code. Test the behavior.

Test 1

`ENVIRONMENT = "development"`

Verify:

```
environment = development
top_k = 5
logging = INFO
```

Test 2

`ENVIRONMENT = "testing"`

Verify:

```
environment = testing
top_k = 3
```

Test 3

`ENVIRONMENT = "production"`

Verify:

```
environment = production
top_k = 10
logging = WARNING
```

Test 4 — deliberately break it

Change:

```
ENVIRONMENT = "production123"
```

Run the application.

You should get a clear:

```
FileNotFoundError
```

because:

```
config/production123.yaml
```

doesn't exist.

That's another useful production-grade behavior: fail early when the requested environment doesn't exist.

One architectural point to remember

We are deliberately keeping:

```
base.yaml
```

for common configuration, and:

```
development.yaml
testing.yaml
production.yaml
```

for differences between environments.

Don't duplicate the entire configuration in all three environment files.

For example, don't do this:

```
development.yaml → 50 lines
testing.yaml     → 50 lines
production.yaml  → 50 lines
```

Instead:

```
base.yaml        → common configuration
development.yaml → only development differences
testing.yaml     → only testing differences
production.yaml  → only production differences
```

This is the DRY (Don't Repeat Yourself) principle applied to configuration.
And that is exactly the pattern we'll build upon in Stage 3.4 — Configuration Overrides and dynamic environment selection, where you won't have to edit settings.py just to switch from development to production.